

Laboratório — Aula 06

Disciplina: Banco de Dados I — Bacharelado em Sistemas de Informação **Tema:** SQL DDL — criar o banco da clínica Patas do Cerrado **Professor:** Me. Alysson Martins Bruno **Tempo estimado:** 1h30min

Antes de começar

Onde fazer o laboratório. Há dois caminhos, e o resultado é o mesmo:

Caminho	Quando usar
No navegador , pelo ambiente preparado da aula: https://labs.iximiuz.com/playgrounds/BD1-AULA06-9e9ee739	não tem Podman instalado, usa um computador do laboratório ou uma máquina fraca
No seu computador , com o Podman da Aula 04	já tem o ambiente montado e quer guardar o trabalho na sua máquina

No ambiente do navegador, entre na sua conta do iximiuz Labs e clique em **Start Playground**; a máquina sobe em menos de um minuto. Ela já vem com a imagem do MySQL 8.4 baixada — o passo 1.2 fica instantâneo — e com os arquivos da aula em `~/lab06`:

Arquivo em <code>~/lab06</code>	Para que serve
<code>compose.yaml</code>	alternativa aos passos 1.1 e 1.2: <code>podman compose up -d</code>
<code>ddl-clinica.sql</code>	script completo da clínica — abra depois de escrever o seu, para comparar
<code>conferir-clinica.sql</code>	mostra o que você criou; use na Parte 4

Duas teclas que ajudam no terminal do navegador: **Ctrl + Shift + V** cola um comando, e a aba **IDE** abre um editor de texto para quem preferir escrever o script antes de executar. Ao encerrar, escolha **Stop** (guarda o ambiente para você retomar) em vez de **Destroy** (apaga tudo).

O que você precisa ter: o ambiente do navegador **ou** o Podman instalado (Aula 04), e o texto base desta aula aberto ao lado, para consultar quando algo não fizer sentido.

Como este roteiro funciona: cada passo traz o comando, a resposta esperada e uma conferência (✓). Só avance quando a conferência do passo atual estiver satisfeita. Se aparecer uma mensagem de erro, procure o número dela na seção **Quando der errado**, ao final deste roteiro.

Quanto tempo reservar: faça as Partes 1 a 4 de uma vez só, sem interromper. A Parte 5 (biblioteca) pode ficar para outro dia.

Parte 1 — Subir o servidor

Passo 1.1 — Rede e volume

Só na primeira vez que você faz isso no computador:

```
podman network create rede-bd1
podman volume create dados-mysql
```

Se já existirem, o Podman avisa que o nome está em uso — e está tudo certo, siga adiante.

Passo 1.2 — O servidor

Se o contêiner ainda não existe:

```
podman run -d \
  --name mysql-bd1 \
  -e MYSQL_ROOT_PASSWORD=minhasenha \
  -e MYSQL_DATABASE=bd1 \
  -p 3306:3306 \
  -v dados-mysql:/var/lib/mysql \
  --network rede-bd1 \
  mysql:8.4
```

Se ele já existe (de outra aula):

```
podman start mysql-bd1
```

Com o arquivo `compose.yaml` desta aula, a alternativa é uma linha só:

```
podman compose up -d
```

Não tem o arquivo? Crie um `compose.yaml` em uma pasta qualquer, com este conteúdo:

```
services:
  mysql:
    image: mysql:8.4
    container_name: mysql-bd1
    environment:
      MYSQL_ROOT_PASSWORD: minhasenha
      MYSQL_DATABASE: bd1
    ports:
      - "3306:3306"
    volumes:
      - dados-mysql:/var/lib/mysql
    networks:
      - rede-bd1

volumes:
  dados-mysql:

networks:
  rede-bd1:
```

O comando `podman compose up -d` precisa ser executado **na pasta onde o arquivo está**.

Passo 1.3 — Conferir

```
podman ps
```

✓ **Conferência:** o contêiner `mysql-bd1` aparece na lista, com estado `Up`. Na primeira execução, o MySQL leva de 20 a 60 segundos para ficar pronto; acompanhe com `podman logs mysql-bd1` até ver a linha `ready for connections`.

Passo 1.4 — Entrar no cliente

```
podman exec -it mysql-bd1 mysql -u root -p
```

Senha: minhasenha.

✓ **Conferência:** a tela mostra o prompt `mysql>`.

Lembre-se: todo comando termina com `;`. Se aparecer `->`, falta fechar o comando. Para cancelar, digite `\c`.

Parte 2 — Criar o banco

Passo 2.1

```
CREATE DATABASE IF NOT EXISTS clinica_veterinaria
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;
```

Query OK, 1 row affected (0.01 sec)

Passo 2.2

```
USE clinica_veterinaria;
SELECT DATABASE();
```

✓ **Conferência:** a resposta é `clinica_veterinaria`.

```
+-----+
| DATABASE() |
+-----+
| clinica_veterinaria |
+-----+
```

Não pule esta conferência. Criar tabelas no banco errado é o engano mais comum — e o mais difícil de perceber.

Parte 3 — Criar as seis tabelas

A ordem importa: primeiro as tabelas sem chave estrangeira.

Passo 3.1 — tutor

```
CREATE TABLE tutor (
  id          BIGINT UNSIGNED AUTO_INCREMENT,
  nome        VARCHAR(100)    NOT NULL,
  telefone    VARCHAR(20)     NOT NULL,
  email       VARCHAR(150),
  CONSTRAINT pk_tutor          PRIMARY KEY (id),
  CONSTRAINT uq_tutor_email    UNIQUE (email)
);
```

✓ **Conferência:** Query OK, 0 rows affected. O 0 rows está certo — DDL cria estrutura, não linhas.

Passo 3.2 — veterinario

```
CREATE TABLE veterinario (  
  id          BIGINT UNSIGNED AUTO_INCREMENT,  
  nome        VARCHAR(100)  NOT NULL,  
  crmv        VARCHAR(20)   NOT NULL,  
  especialidade VARCHAR(80),  
  CONSTRAINT pk_veterinario PRIMARY KEY (id),  
  CONSTRAINT uq_veterinario_crmv UNIQUE (crmv)  
);
```

Passo 3.3 — procedimento

```
CREATE TABLE procedimento (  
  id          BIGINT UNSIGNED AUTO_INCREMENT,  
  nome        VARCHAR(100)  NOT NULL,  
  descricao   VARCHAR(255),  
  valor_referencia DECIMAL(10,2) NOT NULL,  
  CONSTRAINT pk_procedimento PRIMARY KEY (id),  
  CONSTRAINT uq_procedimento_nome UNIQUE (nome),  
  CONSTRAINT ck_procedimento_valor CHECK (valor_referencia >= 0)  
);
```

✓ **Conferência:** SHOW TABLES; mostra três tabelas.

Passo 3.4 — animal

```
CREATE TABLE animal (  
  id          BIGINT UNSIGNED AUTO_INCREMENT,  
  tutor_id    BIGINT UNSIGNED NOT NULL,  
  nome        VARCHAR(80)    NOT NULL,  
  especie     VARCHAR(40)    NOT NULL,  
  raca        VARCHAR(60),  
  data_nascimento DATE,  
  CONSTRAINT pk_animal PRIMARY KEY (id),  
  CONSTRAINT fk_animal_tutor FOREIGN KEY (tutor_id)  
    REFERENCES tutor (id)  
    ON DELETE RESTRICT  
    ON UPDATE CASCADE  
);
```

Passo 3.5 — consulta

```
CREATE TABLE consulta (  
  id          BIGINT UNSIGNED AUTO_INCREMENT,  
  animal_id   BIGINT UNSIGNED NOT NULL,  
  veterinario_id BIGINT UNSIGNED NOT NULL,  
  data_hora   DATETIME        NOT NULL,  
  motivo      VARCHAR(255)    NOT NULL,  
  observacoes TEXT,  
  CONSTRAINT pk_consulta PRIMARY KEY (id),  
  CONSTRAINT fk_consulta_animal FOREIGN KEY (animal_id)  
    REFERENCES animal (id)  
    ON DELETE RESTRICT  
    ON UPDATE CASCADE,  
  CONSTRAINT fk_consulta_veterinario FOREIGN KEY (veterinario_id)  
    REFERENCES veterinario (id)  
    ON DELETE RESTRICT  
    ON UPDATE CASCADE,  
  CONSTRAINT uq_consulta_agenda UNIQUE (veterinario_id, data_hora)  
);
```

Passo 3.6 — consulta_procedimento

```
CREATE TABLE consulta_procedimento (  
  id          BIGINT UNSIGNED AUTO_INCREMENT,  
  consulta_id BIGINT UNSIGNED NOT NULL,  
  procedimento_id BIGINT UNSIGNED NOT NULL,  
  quantidade  SMALLINT UNSIGNED NOT NULL DEFAULT 1,  
  valor_praticado DECIMAL(10,2) NOT NULL,  
  CONSTRAINT pk_consulta_procedimento PRIMARY KEY (id),  
  CONSTRAINT fk_cp_consulta FOREIGN KEY (consulta_id)  
    REFERENCES consulta (id)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE,  
  CONSTRAINT fk_cp_procedimento FOREIGN KEY (procedimento_id)  
    REFERENCES procedimento (id)  
    ON DELETE RESTRICT  
    ON UPDATE CASCADE,  
  CONSTRAINT uq_consulta_procedimento UNIQUE (consulta_id, procedimento_id),  
  CONSTRAINT ck_cp_quantidade CHECK (quantidade > 0),  
  CONSTRAINT ck_cp_valor CHECK (valor_praticado >= 0)  
);
```

✓ Conferência final da Parte 3:

```
SHOW TABLES;
```

```
+-----+
| Tables_in_clinica_veterinaria |
+-----+
| animal                        |
| consulta                     |
| consulta_procedimento        |
| procedimento                 |
| tutor                        |
| veterinario                  |
+-----+
6 rows in set (0.00 sec)
```

Seis tabelas. Chegou até aqui? A parte mais importante do laboratório está feita.

Parte 4 — Conferir e testar

Passo 4.1 — Ler a estrutura

```
DESCRIBE animal;
```

✓ **Conferência:** apenas `raca` e `data_nascimento` aparecem com `Null = YES`. Todas as outras colunas são obrigatórias. `id` tem `PRI` e `auto_increment`; `tutor_id` tem `MUL`.

Repita o `DESCRIBE` nas outras cinco tabelas e compare com o modelo lógico da Aula 05.

Passo 4.2 — Ver a tabela inteira

```
SHOW CREATE TABLE consulta\G
```

✓ **Conferência:** as duas chaves estrangeiras aparecem com os nomes `fk_consulta_animal` e `fk_consulta_veterinario`, e a unicidade `uq_consulta_agenda` está lá.

Passo 4.3 — Listar todas as restrições

```
SELECT table_name, constraint_name, constraint_type
FROM information_schema.table_constraints
WHERE table_schema = 'clinica_veterinaria'
ORDER BY table_name, constraint_type;
```

No ambiente do navegador, o mesmo relatório sai pronto e mais completo com:

```
SOURCE /home/laborant/lab06/conferir-clinica.sql;
```

Ele lista as seis tabelas esperadas (marcando `>>> FALTA` no que não foi criado), as colunas com a obrigatoriedade de cada uma, as restrições, as chaves estrangeiras com as regras `ON DELETE/ON UPDATE`

e os CHECK.

✓ **Conferência:** aparecem as chaves primárias (sempre com o nome PRIMARY), as estrangeiras, as de unicidade e as de verificação (CHECK) de todas as tabelas.

Passo 4.4 — Tentar quebrar as regras

Os comandos abaixo **devem falhar**. Se algum funcionar, alguma restrição ficou faltando.

```
-- 1. tutor que não existe
INSERT INTO animal (tutor_id, nome, especie) VALUES (999, 'Rex', 'Canino');
```

Esperado: ERROR 1452 ... a foreign key constraint fails

```
-- 2. coluna obrigatória em branco
INSERT INTO tutor (telefone) VALUES ('63999990000');
```

Esperado: ERROR 1364 ... Field 'nome' doesn't have a default value

```
-- 3. valor negativo
INSERT INTO procedimento (nome, valor_referencia) VALUES ('Consulta', -50);
```

Esperado: ERROR 3819 ... Check constraint 'ck_procedimento_valor' is violated.

```
-- 4. CRMV repetido (o primeiro funciona, o segundo não)
INSERT INTO veterinario (nome, crmv) VALUES ('Ana Souza', 'T0-1234');
INSERT INTO veterinario (nome, crmv) VALUES ('Ana S. Souza', 'T0-1234');
```

Esperado: ERROR 1062 ... Duplicate entry 'T0-1234'

Limpe o registro de teste:

```
DELETE FROM veterinario WHERE crmv = 'T0-1234';
```

✓ **Conferência:** quatro erros nos quatro testes. Cada erro é uma regra do negócio funcionando.

Passo 4.5 — Corrigir sem recomeçar

Acrescente uma coluna, confira e desfaça:

```
ALTER TABLE tutor ADD COLUMN cidade VARCHAR(60);
DESCRIBE tutor;
ALTER TABLE tutor DROP COLUMN cidade;
DESCRIBE tutor;
```

✓ **Conferência:** a coluna aparece no primeiro DESCRIBE e some no segundo.

Passo 4.6 — Ver a ordem de exclusão funcionando

```
DROP TABLE tutor;
```

Esperado: ERROR 3730 (HY000): Cannot drop table 'tutor' referenced by a foreign key constraint...

✓ **Conferência:** o erro apareceu — e a tabela **não** foi apagada. Confirme com `SHOW TABLES;`.

Parte 5 — Desafio: a biblioteca comunitária

Agora sem passo a passo. Implemente, no banco biblioteca, o modelo que você construiu na Aula 05:

```
LEITOR (id PK, nome, telefone)
LIVRO (id PK, titulo, isbn UQ)
EXEMPLAR (id PK, livro_id FK, codigo_tombo UQ)
EMPRESTIMO (id PK, leitor_id FK, exemplar_id FK,
            data_retirada, data_devolucao_prevista,
            data_devolucao_real)
```

Exigências:

1. banco criado com utf8mb4;
2. as quatro tabelas, na ordem correta de dependência;
3. id BIGINT UNSIGNED AUTO_INCREMENT com PRIMARY KEY em todas;
4. NOT NULL em nome, titulo, codigo_tombo, data_retirada e data_devolucao_prevista;
5. UNIQUE em isbn e em codigo_tombo;
6. chaves estrangeiras com ON DELETE RESTRICT ON UPDATE CASCADE;
7. data_devolucao_real opcional.

Conferência do desafio:

```
USE biblioteca;
SHOW TABLES;           -- quatro tabelas
DESCRIBE emprestimo;     -- só data_devolucao_real com Null = YES
SHOW CREATE TABLE exemplar\G  -- a FK aponta para livro (id)
```

Salve o script em um arquivo chamado biblioteca.sql. Ele é o comprovante do seu trabalho e o ensaio do init.sql da A2.

Encerrando a sessão

```
exit
```

```
podman stop mysql-bd1
```

Os dados continuam no volume dados-mysql: na próxima vez, `podman start mysql-bd1` devolve tudo como estava. **Não execute `podman volume rm dados-mysql`** — isso apagaria o seu trabalho.

Quando der errado

Mensagem	Causa provável	O que fazer
ERROR 1046 ... No database selected	faltou o USE	USE clinica_veterinaria;
ERROR 1064 ... syntax	vírgula sobrando antes do), parêntese aberto, palavra digitada errada	olhe o trecho citado e a linha anterior
ERROR 1050 ... already exists	a tabela já foi criada	SHOW TABLES; para confirmar e seguir adiante
ERROR 1824 ... Failed to open the referenced table	a tabela pai ainda não existe	crie a tabela pai e repita o comando
ERROR 3780 ... are incompatible	tipo da FK diferente do tipo da PK	iguale os tipos, inclusive UNSIGNED
ERROR 3730 ... Cannot drop table	há FK apontando para a tabela	apague antes as tabelas filhas
ERROR 1045 ... Access denied	senha errada	a senha é minhasenha
Can't connect to MySQL server	o servidor ainda está iniciando ou está parado	podman ps e podman logs mysql-bd1
o prompt mostra ->	comando incompleto	; para fechar, ou \c para cancelar

Precisa recomeçar do zero? Apague na ordem inversa:

```
DROP TABLE IF EXISTS consulta_procedimento;  
DROP TABLE IF EXISTS consulta;  
DROP TABLE IF EXISTS animal;  
DROP TABLE IF EXISTS procedimento;  
DROP TABLE IF EXISTS veterinario;  
DROP TABLE IF EXISTS tutor;
```

Checklist de conclusão

- ☐ o servidor MySQL sobe e eu chego ao prompt mysql>
- ☐ SELECT DATABASE() responde clinica_veterinaria
- ☐ SHOW TABLES mostra as seis tabelas
- ☐ DESCRIBE de cada tabela confere com o modelo lógico da Aula 05
- ☐ os quatro testes da Parte 4 falharam como esperado

- ☐ sei apagar e recriar uma tabela na ordem certa
- ☐ o script `biblioteca.sql` está escrito e testado
- ☐ respondi ao questionário da Aula 06 no Google Classroom